多平台向量GEMM实现对比与RVV扩展指令建议

2026-04-14

目 录

各平台关键发现
RVV扩展指令建议总结
YOLO+ONNX Runtime场景收益分析
实现路线图
附录：各平台指令详细对比
结论

概述

本文档基于ONNX Runtime MLAS库中各平台架构的SGEMM/FGEMM内核实现进行对比分析，目标是识别对RVV（RISC-V向量扩展）有借鉴价值的关键指令模式，并提出针对YOLO+ONNX Runtime场景的RVV扩展指令建议。

分析平台：AVX/AVX2、ARM NEON、LoongArch LASX/LSX、Power VSX (POWER10)、S390X Z-Vector、WASM SIMD

基准实现：RVV VL=16内核（512位VLEN，SEW=32，LMUL=1）

各平台关键发现

1. x86 AVX/AVX2

核心特点：

- YMM寄存器（256位，8个float32）
- 无掩码寄存器（AVX/AVX2限制，AVX512才有k0-k7掩码）
- 丰富的数据重排指令（vunpck、vperm、vshuf系列）

高价值指令：

指令	功能	RVV现状
`vunpcklps/vunpckhps`	交错解包（低/高位元素分离）	RVV无直接对应，需用vrgather模拟
`vpermilps`	立即数索引置换（编译时固定模式）	`vrgather.vi`支持单元素立即数索引，但不支持完整模式掩码
`vshufps`	跨寄存器元素选择	vrgather可实现但效率较低

收益分析：矩阵转置是GEMM的核心操作（A矩阵行转列、B矩阵列打包）。AVX使用`vunpck`指令树实现4×4转置仅需6条指令（注意使用独立目标寄存器避免写后读冲突）：

```
; 4×4 float矩阵转置（AVX，正确用法）
vunpcklps ymm4, ymm0, ymm1 ; 交错低半部: [a0,b0,a1,b1, ...]
vunpckhps ymm5, ymm0, ymm1 ; 交错高半部: [a2,b2,a3,b3, ...]
vunpcklps ymm6, ymm2, ymm3
vunpckhps ymm7, ymm2, ymm3
vperm2f128 ymm0, ymm4, ymm6, 0x20 ; 跨128位段重组
vperm2f128 ymm1, ymm4, ymm6, 0x31
...
```

RVV实现4×4转置需要4-6条`vrgather`指令（每行一条），加上索引向量准备（可预计算复用），总计约6-8条指令。RVV的优势是`vrgather`提供完全任意的重排模式，不限于固定交织方式。

建议扩展：

- `vunzip.vv`：奇偶元素分离（等效vunpck）

- `vzip.vv`：奇偶元素交错合并
- `vshuffle.vx`：立即数模式置换（编译时固定）

2. ARM NEON

核心特点：

- 128位Q寄存器（4个float32）
- Lane-indexed操作（直接引用向量中特定lane）
- 跨寄存器交换指令（vtrn/vzip/vuzp）

高价值指令：

指令	功能	RVV现状
`vmlaq_lane_f32`	Lane-indexed FMA：`q0[i] += q_b[i] * v_a[lane]`	RVV无等效，需逐标量加载广播
`vtrn/vzip/vuzp`	跨寄存器元素交换/解包	vrgather模拟，效率低
`vld1q_lane_f32`	单元素加载到特定lane	RVV需load+insert两步

收益分析： Lane-indexed FMA在GEMM内核中的核心价值是批量加载**A**元素并逐个广播。MLAS ARM64内核实际使用模式：

```
; NEON实际GEMM模式（来自MLAS SgemmKernelNeon.asm）
ldr d0, [x1], #8 ; 加载4个A元素到v0
fmla v4.4s, v8.4s, v0.s[0] ; 用v0的lane[0]广播，与B向量FMA
fmla v4.4s, v9.4s, v0.s[1] ; 用v0的lane[1]广播
fmla v4.4s, v10.4s, v0.s[2]; 用v0的lane[2]广播
fmla v4.4s, v11.4s, v0.s[3]; 用v0的lane[3]广播
; 一次加载A，4条指令完成4个K步的FMA，无需额外广播指令
```

当前RVV实现（标量广播模式）：

```
; RVV当前：逐个加载A标量并广播
flw fa0, 0(a1) ; 加载A[0]
vfmacc.vf v_acc, fa0, v_b ; FMA（内部广播fa0）
flw fa0, 4(a1) ; 加载A[1]
vfmacc.vf v_acc, fa0, v_b+16 ; FMA
; 每个K步需要1条load + 1条FMA = 2条指令
; NEON: 1条load + 4条lane-FMA = 5条指令完成4个K步，平均1.25条/K步
```

Lane-indexed FMA的核心优势是减少**A**矩阵加载指令数（4个元素合并为一次load），而非减少FMA指令数。

建议扩展：

- `vfmacc_lane.vf vd, vs1, vs2, lane`：Lane-indexed FMA
- 功能：`vd[i] += vs2[i] * vs1[lane]`（广播vs1的指定lane，与vs2逐元素FMA）
- 核心价值：A矩阵4元素合并加载，逐lane广播FMA，减少K循环加载指令75%
- 应用场景：多行GEMM、卷积im2col

3. LoongArch LASX/LSX

核心特点：

- LASX：256位X寄存器（8个float32）
- LSX：128位L寄存器（4个float32）
- 复合操作指令（加载+广播、置换+选择）

高价值指令：

指令	功能	RVV现状
`xvldrepl.w`	加载单元素并广播到整个向量	RVV可通过`vlse32.v` stride=0实现等效功能（见下文）
`xvpermi.w`	立即数索引置换（4元素）	vrgather需要向量索引
`xvreplgr2vr.w`	GR到VR广播	vfmv.v.f等效

收益分析： `xvldrepl.w`将加载和广播合并为单指令：

```
; LoongArch: 1条指令完成load+broadcast
xvldrepl.w xr0, a0, 0 ; 从内存加载一个float32，广播到xr0[0..7]
```

RVV已有等效能力：RVV规范支持通过strided load实现load+broadcast：

```
; RVV: 1条指令 — strided load with stride=0
; RVV规范明确说明: "If the stride value is 0, only the element at the
; address in rs1 is accessed, and its value is copied to all active elements"
vlse32.v v0, (a0), x0 ; 加载float32，stride=0，广播到v0所有元素
; C intrinsics: __riscv_vlse32_v_f32m1(addr, 0, vl)
```

因此LoongArch的`xvldrepl.w`优势在RVV中已原生存在，无需额外扩展。

> 注：`vlse32.v` stride=0在功能上等效，但不同微架构实现可能有性能差异。专用load+broadcast指令在某些硬件上可能更高效（避免stride计算逻辑），但这属于微架构优化范畴。

4. Power VSX (POWER10)

核心特点：

- 128位VSX寄存器（可拼接为256位）
- POWER10 MMA（Matrix Multiply Assist）指令集
- 外积计算指令

最高价值指令：

指令	功能	RVV现状
`xvf32gerpp`	4×4矩阵外积累加：`ACC += A[4] × B[4]^T`	RVV无矩阵级指令
`xvf32gernp`	带负号的外积累加	无
`xxmfacc/xxmtacc`	accumulator到寄存器移动	无accumulator概念

收益分析： MMA指令是所有分析平台中最具颠覆性的特性。单条`xvf32gerpp`完成16个乘加操作：

```
; POWER10: 单指令完成4×4矩阵乘法累加
xvf32gerpp acc0, vA, vB ; acc0[4×4] += vA[0..3] × vB[0..3]^T
; 等效于16次: acc[i,j] += A[i] * B[j]
```

传统向量实现需要：

```
; RVV VL=16（当前实现，K展开=2，处理2行×16列）：
; 每次K迭代（2个K步）：
vfmacc.vf v_acc_r0, a0_r0, v_b0 ; row0 += A[0] * B[0..15]
vfmacc.vf v_acc_r1, a0_r1, v_b0 ; row1 += A[0] * B[0..15] (ProcessTwoRows)
vfmacc.vf v_acc_r0, a1_r0, v_b1 ; row0 += A[1] * B[16..31]
vfmacc.vf v_acc_r1, a1_r1, v_b1 ; row1 += A[1] * B[16..31]
; 4条vfmacc处理 2行 × 16列 × 2K = 64个乘加

; POWER10 MMA（K展开=4，处理2行×16列）：
; 每次K迭代（4个K步）：
xvf32gerpp acc[0..3], ABroadcast, BElements[0..3] ; row0 × 16列 × 4K
xvf32gerpp acc[4..7], A2Broadcast, BElements[0..3] ; row1 × 16列 × 4K
; 8条MMA处理 2行 × 16列 × 4K = 128个乘加
; 但MMA需要额外的disassemble_acc开销
```

注：指令数对比需归一化到相同工作量。每K步每行每16列：RVV需2条vfmacc，POWER10需2条MMA，原始吞吐量相当。MMA的优势主要来自专用accumulator减少寄存器压力。

性能提升预估（基于实际**MLAS**内核对比）：

- POWER10 MMA每次K迭代处理4×16=64个FMA（4个B块 × 4×4外积），需要4条MMA指令
- RVV VL=16每次K迭代（2个K步）处理2×2×16=64个FMA，需要4条`vfmacc.vf`指令
- 指令数相当，但MMA的优势在于：
- 寄存器压力降低：专用accumulator寄存器（POWER10有8个`__vector_quad`），不占用向量寄存器
- K展开宽度：MMA天然处理4个K元素的外积，RVV当前K展开为2
- B加载共享：4行A共享同一次B加载（外积特性）
- 实际性能提升预估：**1.3-2.0x**（来自寄存器压力降低和更宽的K展开，而非原始吞吐量差异）

建议扩展：

- `vmulacc.vv vd, vs1, vs2`：矩阵乘累加（外积）
- 功能：`vd[4×4] += vs1[0..3] × vs2[0..3]^T`
- 适用VLEN128（SEW=32，LMUL1）
- 最高优先级扩展

5. S390X Z-Vector

核心特点：

- 128位VR寄存器（4个float32）
- 常量掩码置换指令
- 内存预取指令

高价值指令：

指令	功能	RVV现状
`vec_perm`	常量掩码置换（编译器可优化）	vrgather需运行时向量索引
`vlebrf`	加载并字节序交换	RVV无字节序处理指令
`pfd`	预取指令（数据/指令分离）	RVV无预取指令

收益分析：`vec_perm`使用向量寄存器中的常量字节掩码，编译器可在编译时预计算掩码并优化：

```
// S390X实际代码（来自MLAS SgemmKernelZVECTOR.cpp）
const __vector unsigned char mask_even = { 0, 1, 2, 3, 16, 17, 18, 19, 8, 9, 10, 11, 24, 25, 26,
27 };
a1 = vec_perm(AElements[0], AElements[1], mask_even); // 3操作数: dst, src1, src2, mask_reg
```

RVV的`vrgather`需要向量索引：

```
; RVV: 需准备索引向量（运行时开销）
vid.v v_idx ; 生成索引 [0,1,2,3,...]
vrgather.vv v0, v1, v_idx ; 运行时置换
```

两者的关键区别：S390X的mask是16字节的查找表（每个字节索引一个元素），而RVV的vrgather使用元素级索引。对于编译时常量模式，RVV可以将索引向量预计算并存储在`.rodata`段，运行时仅需一条`vl`加载索引 + 一条`vrgather`，与S390X的"加载mask + vec_perm"效率相当。

建议扩展：

- `vperm.vx vd, vs2, imm`：立即数掩码置换（编译时固定）
- `vprefetch.r addr`：数据预取指令

6. WASM SIMD

核心特点：

- 128位v128寄存器（4个float32）
- 极简指令集（仅基础操作）
- 无原生FMA（需mul+add组合）

对比结论： RVV在几乎所有方面优于WASM SIMD：

- RVV原生FMA vs WASM `f32x4.mul + f32x4.add`
- RVV动态VL vs WASM固定128位
- RVV丰富的数据操作 vs WASM极简置换

无扩展建议：WASM SIMD无值得借鉴的高级特性。

RVV扩展指令建议总结

按优先级排序

优先级	扩展指令	来源平台	预期收益	实现难度	备注
P0	`vmulacc.vv` (矩阵外积)	Power VSX	1.3-2.0x性能提升	高（需新硬件单元）	寄存器压力和K展开收益
P1	`vfmacc_lane.vf` (Lane-indexed FMA)	ARM NEON	减少A加载指令75%	中	批量加载A元素
~~P2~~	~~`vlw.repl.v` (Load+Broadcast)~~	~~LoongArch~~	~~已有等效~~	—	**RVV**已支持（`vlse32.v` stride=0）
P2	`vunzip/vzip.vv` (奇偶分离/合并)	AVX/NEON	优化转置25-40%	中	
P3	`vperm.vx` (立即数置换)	S390X/AVX	编译时优化	低	编译器可预计算索引
P4	`vprefetch.r` (预取)	S390X	内存延迟优化	低	

P0级：矩阵乘累加指令

指令定义：

```
vmulacc.vv vd, vs1, vs2
功能：vd[4×4矩阵] += vs1[0..3] × vs2[0..3]^T（外积）
要求：SEW=32, LMUL≥1（支持4×4矩阵）
```

应用场景：

- GEMM内核：单指令完成K=4块的计算
- 卷积：im2col后的矩阵乘加速
- 深度学习：全连接层前向传播

硬件实现考量：

- 需要4×4矩阵accumulator寄存器（类似POWER10）

- 可复用现有FMA硬件单元并行化
- VL动态控制需适配矩阵维度

P1级：Lane-indexed FMA

指令定义：

```
vfmacc_lane.vf vd, rs1, vs2, lane[imm]
功能：vd[i] += rs1 * vs2[lane] (i = 0..VL-1)
要求：lane为立即数（0..VL-1范围内）
```

应用场景：

- 多行GEMM：批量加载A矩阵元素到向量，逐lane广播FMA
- 卷积核：权重矩阵特定通道复用
- 减少A矩阵加载指令数（4个元素合并为1次向量load）

示例代码对比：

```
; 当前RVV实现（逐标量加载+广播FMA）
flw fa0, 0(a1) ; 加载A[0]（标量）
vfmacc.vf v_acc0, fa0, v_b ; FMA
flw fa0, 4(a1) ; 加载A[1]
vfmacc.vf v_acc0, fa0, v_b+16 ; FMA
; 4个K步 = 8条指令（4 load + 4 FMA）

; 扩展后（Lane-indexed FMA，参照NEON模式）
vle32.v v_a, (a1) ; 一次加载4个A元素
vfmacc_lane.vf v_acc0, v_a, v_b0, 0 ; 用v_a[0]广播FMA
vfmacc_lane.vf v_acc0, v_a, v_b1, 1 ; 用v_a[1]广播FMA
vfmacc_lane.vf v_acc0, v_a, v_b2, 2 ; 用v_a[2]广播FMA
vfmacc_lane.vf v_acc0, v_a, v_b3, 3 ; 用v_a[3]广播FMA
; 4个K步 = 5条指令（1 load + 4 lane-FMA），减少37.5%
```

~~P2级：Load+Broadcast~~ （已存在等效功能）

RVV已原生支持，无需扩展：

```
vlse32.v vd, (rs1), x0 ; stride=0 → load + broadcast
; 等效于：flw ft0, (rs1); vfmv.v.f vd, ft0
; 但仅需1条指令
```

RVV规范明确：stride为0时，仅访问rs1地址的单个元素并复制到所有活跃元素。

此需求可从扩展列表中移除。

P2级：奇偶分离/合并（原P3）

指令定义：

```
vunzip.vv vd, vs2
功能：将vs2奇偶元素分离，偶数元素放vd低位，奇数元素放vd高位

vzip.vv vd, vs2
功能：将vs2低/高位元素交错合并
```

应用场景：

- 矩阵转置：替代vrgather实现
- 数据重排：FFT、卷积数据布局变换

P3级：立即数置换（原P4）

指令定义：

```
vshuffle.vx vd, vs2, imm[8-bit]
功能：根据立即数掩码置换vs2元素到vd
```

应用场景：

- 编译时已知置换模式（如转置）
- 减少运行时索引向量开销

YOLO+ONNX Runtime场景收益分析

指令热点分析

基于QEMU BBV profiling，YOLO推理中热点函数分布：

函数	占比	关键操作
`MlasSgemmKernel`	35%	K循环FMA、元素广播
`Convolution`	25%	im2col、矩阵乘
`BatchNorm`	15%	元素级向量运算
`Activation`	10%	ReLU向量比较
其他	15%	数据搬运、后处理

扩展指令收益估算

针对GEMM内核（35%热点）：

扩展指令	当前指令数	扩展后指令数	减少比例
`vmulacc.vv` (P0)	4条FMA/K=4块	1条MMA	75%指令减少，实际性能提升1.3-2.0x
`vfmacc_lane.vf` (P1)	8条(4 load + 4 FMA)/4K步	5条(1 load + 4 lane-FMA)/4K步	37.5%
~~`vlw.repl.v`~~	~~2条~~	~~1条~~	**RVV**已支持（vlse32 stride=0）

综合收益估算：

- P0实现：GEMM内核性能提升**1.3-2.0x**（来自寄存器压力降低和K展开），整体推理加速**1.1-1.5x**
- P1：GEMM内核K循环指令数减少**37.5%**
- P2（vunzip/vzip）优化转置：数据预处理阶段加速**25-40%**

实现路线图

阶段1：软件模拟验证（QEMU扩展）

1. 在QEMU中添加扩展指令模拟
2. 修改MLAS内核使用新指令
3. 对比原版与扩展版性能

阶段2：硬件设计建议

1. 指令编码方案（参考RVV编码空间）
2. 硬件单元设计（MMA accumulator）
3. 与现有RVV规范兼容性

阶段3：编译器支持

1. LLVM后端扩展指令支持
2. 内联汇编接口
3. 自动向量化优化

附录：各平台指令详细对比

FMA指令对比

平台	指令	操作数	特点
RVV	`vfmacc.vf`	vec+scalar+vec	标量广播FMA
AVX512	`vfmadd231ps`	zmm+zmm+zmm	三向量FMA
NEON	`vmlaq_lane_f32`	vec+vec+vec[lane]	Lane-indexed（A打包加载，逐lane广播）
LoongArch	`xvfmadd.s`	xr+xr+xr	向量FMA（`xfmad.d`为标量双精度）
Power VSX	`xvf32gerpp`	acc+vec+vec	4×4矩阵外积（需专用accumulator）

数据重排指令对比

平台	交错/解包	置换	转置效率
AVX	vunpck系列	vpermilps/vpermd	6条指令
NEON	vtrn/vzip/vuzp	无立即数置换	4条指令
RVV	vrgather模拟	vrgather.vv/vx	6-8条指令（含索引准备）
S390X	无	vec_perm（立即数）	中等
Power	无	xxpermdi	中等

结论

通过多平台对比分析，识别出以下对RVV最具价值的扩展方向：

1. 矩阵级乘累加指令（**P0**）：借鉴Power VSX MMA，实现单指令4×4矩阵外积。收益主要来自专用accumulator寄存器降低压力和更宽的K展开，预估提升1.3-2.0x
2. **Lane-indexed FMA**（**P1**）：借鉴ARM NEON `fmla ... v.s[lane]`，允许批量加载A元素后逐lane广播FMA，减少K循环中37.5%的指令数
3. **Load+Broadcast**（原**P2**）：~~借鉴LoongArch~~ **RVV**已原生支持（`vlse32.v` stride=0），无需扩展
4. 数据重排指令（**P2**）：借鉴AVX/NEON的vunpck/vtrn，优化矩阵转置效率

建议优先验证P0（矩阵外积）和P1（lane-indexed FMA）的可行性，这两项可在YOLO+ONNX Runtime场景带来实际性能收益。